

Date:

31 January 2026

Team ID:

TEAM-AI-14

Project Name:

AI-Powered Automated Support Ticket Classification System

TECHNOLOGY STACK (ARCHITECTURE & STACK)

Technical Architecture:

The deliverable includes the architectural diagram below, illustrating the complete data flow and component interaction for automated support ticket classification. Tables 1 and 2 detail the individual software components, infrastructure demarcations, third-party integrations, data storage services, and architectural characteristics.

Reference Scenario: AI-powered backend workflow automation for enterprise customer support ticketing systems.
Reference Architecture Inspiration: IBM Cloud Architecture Patterns for Intelligent Processing & AWS Architecture Center.

FIGURE 1: ARCHITECTURAL FLOW DIAGRAM - AUTOMATED TICKET CLASSIFICATION SYSTEM

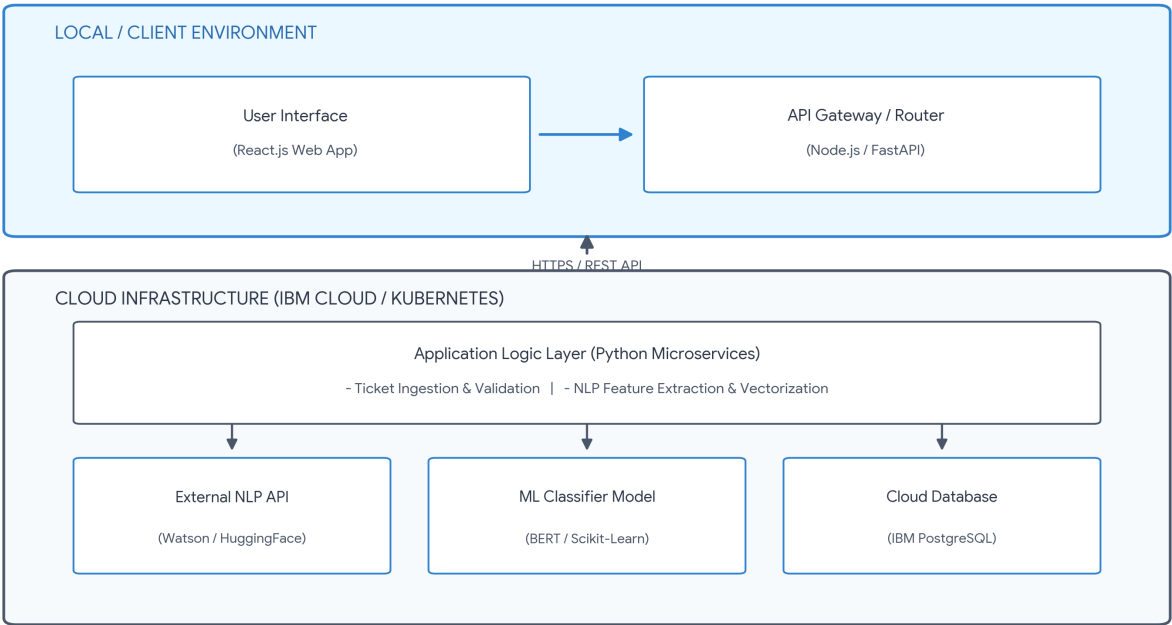


TABLE-1: COMPONENT DESCRIPTION & TECHNOLOGY STACK

S.NO	COMPONENT	DESCRIPTION	TECHNOLOGY
1.	User Interface	How user or support staff interacts with application (Web dashboard for ticket	HTML5, CSS3, JavaScript, React.js, Tailwind CSS

S.NO	COMPONENT	DESCRIPTION	TECHNOLOGY
		submission, tracking, and viewing classification analytics).	
2.	Application Logic-1	Logic for ticket ingestion, validation, sanitization, and RESTful routing of incoming customer support requests.	Python, FastAPI, Pydantic
3.	Application Logic-2	Natural Language Processing (NLP) pipeline for text tokenization, stop-word removal, lemmatization, and sentiment analysis.	Python, NLTK, spaCy, Hugging Face Transformers
4.	Application Logic-3	Automated ticket routing, priority assignment engine, and notification dispatcher to respective department queues.	Celery, Redis Message Broker, Python Worker Nodes
5.	Database	Local development data store for application configurations, user credentials, and offline ticket caching.	PostgreSQL / SQLite
6.	Cloud Database	Managed cloud database service for persistent storage of historical tickets, classification logs, and model feedback loops.	IBM Cloud Databases for PostgreSQL, IBM Cloudant (NoSQL)
7.	File Storage	Storage service for customer-uploaded attachments, ticket screenshots, and exported audit reports.	IBM Cloud Object Storage / AWS S3 Compatible Storage
8.	External API-1	Third-party email integration API for parsing inbound customer support emails into structured classification tickets.	SendGrid Inbound Parse API / Microsoft Graph API
9.	External API-2	Enterprise CRM integration API to fetch customer profile details, SLA tiers, and previous ticket history.	Salesforce REST API / HubSpot CRM API
10.	Machine Learning Model	Multi-class text classification model trained to categorize support tickets by intent, department, and urgency level.	Fine-tuned BERT / DistilBERT transformer model & Scikit-Learn TF-IDF + Logistic Regression baseline
11.	Infrastructure (Server / Cloud)	Application deployment architecture across local testing environments and production cloud infrastructure.	Local: Docker Compose Cloud: IBM Cloud Code Engine, Kubernetes (IKS), Helm Charts

TABLE-2: APPLICATION CHARACTERISTICS

S.NO	CHARACTERISTICS	DESCRIPTION	TECHNOLOGY
1.	Open-Source Frameworks	List of open-source frameworks utilized for backend processing, machine learning, and frontend presentation.	FastAPI, React.js, spaCy, Scikit-Learn, PyTorch, Hugging Face Transformers

S.NO	CHARACTERISTICS	DESCRIPTION	TECHNOLOGY
2.	Security Implementations	Security protocols, authentication, access controls, data encryption in transit and at rest, and vulnerability mitigation.	OAuth 2.0, JSON Web Tokens (JWT), HTTPS/TLS 1.3, SHA-256 password hashing, IAM Roles, OWASP Top 10 guidelines
3.	Scalable Architecture	Justification of scalability using containerized microservices architecture to handle peak ticket volumes dynamically.	Microservices architecture orchestrated via Kubernetes Horizontal Pod Autoscaler (HPA) and Docker containers
4.	Availability	Design measures ensuring high availability (HA) with zero single points of failure across multi-zone cloud deployments.	IBM Cloud Load Balancers, Multi-zone Kubernetes worker nodes, automated health checks, and Redis replication
5.	Performance	Performance optimization strategies including asynchronous request handling, response caching, and fast model inference.	Asynchronous FastAPI endpoints, Redis caching for frequent ticket metadata, CDN for static frontend assets

REFERENCES

- <https://c4model.com/>
- <https://developer.ibm.com/patterns/ai-powered-backend-system-for-order-processing-during-pandemics/>
- <https://www.ibm.com/cloud/architecture>
- <https://aws.amazon.com/architecture>
- <https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d>
- <https://huggingface.co/docs/transformers>